SWCON221 Microservice Programming

Lecture 10

Docker Storage

2026. 09. 01

Sungwon Lee

Department of Software Convergence

경희대학교
KYUNG HEE UNIVERSITY

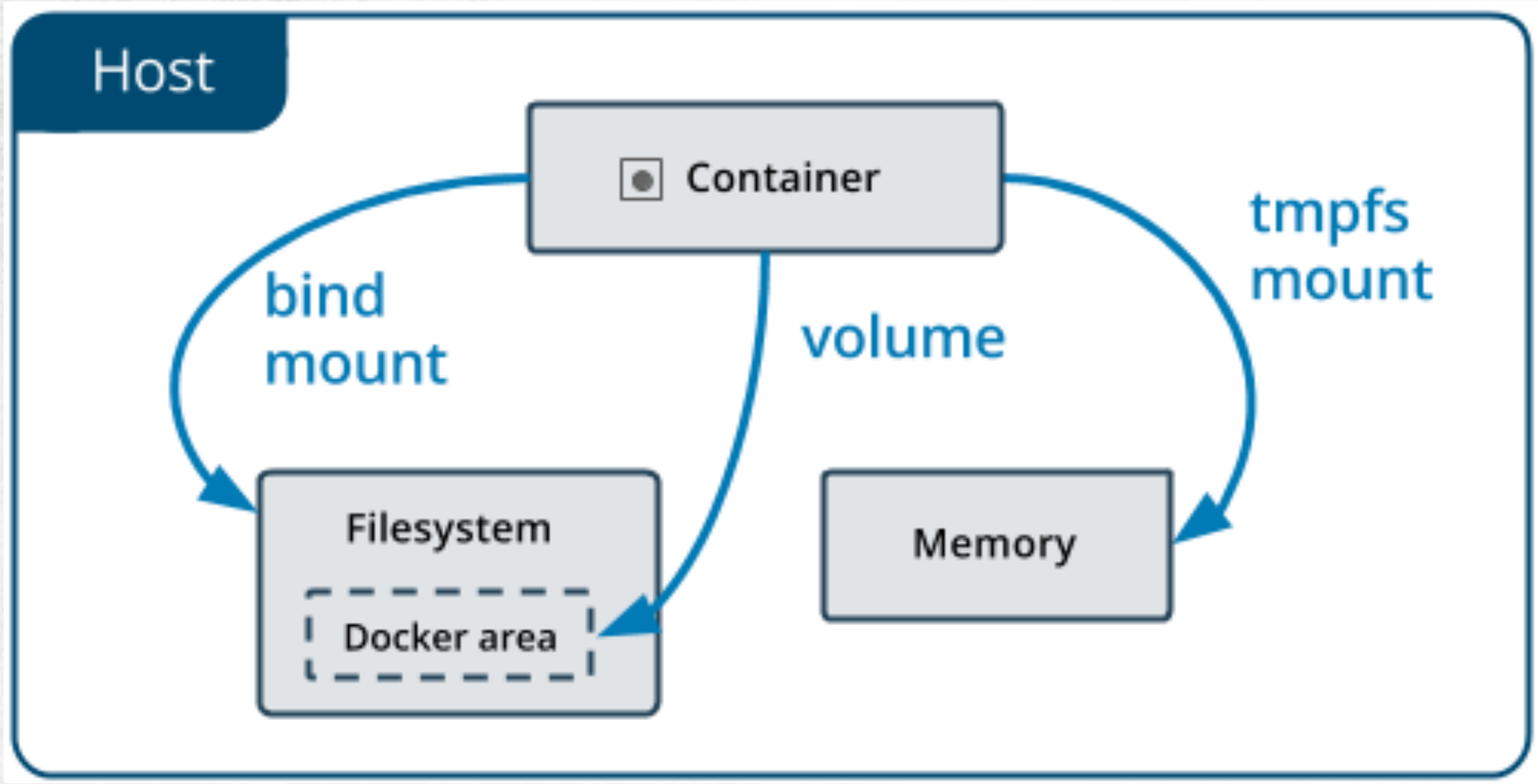
Contents

- Issues
- Storing Files
- Storage Options
  - ◉ volumes
  - ◉ bind mounts
  - ◉ tmpfs mount (for Docker on Linux)
  - ◉ named pipe (for Docker on Windows)
- ✴ Practice

Issues when storing files inside a container

- The data doesn't persist when that container no longer exists, and it can be difficult to get the data out of the container if another process needs it.
- A container's writable layer is tightly coupled to the host machine where the container is running. You can't easily move the data somewhere else.
- Writing into a container's writable layer requires a storage driver to manage the filesystem. The storage driver provides a union filesystem, using the Linux kernel. This extra abstraction reduces performance as compared to using data volumes, which write directly to the host filesystem.

Storing files in the host machine



`Volumes` are stored in a part of the host filesystem which is managed by Docker (/var/lib/docker/volumes/ on Linux)

`Bind mounts` may be stored anywhere on the host system.

`tmpfs mounts` are stored in the host system's memory only, and are never written to the host system's filesystem.

Reference: https://docs.docker.com/storage/

Option1: Volumes (1/2)

- Stored in a part of the host filesystem which is managed by Docker
  - Linux: /var/lib/docker/volumes/
  - Non-Docker processes should not modify this part of the filesystem
  - Volumes are the best way to persist data in Docker
- A new directory is created within Docker's storage directory on the host machine
  - Docker manages that directory's contents.
  - May be named or anonymous

Option1: Volumes (2/2)

- Volume Characteristics:
  - A given volume can be mounted into multiple containers simultaneously
  - When no running container is using a volume, the volume is still available to Docker and is not removed automatically
  - Possible to store your data on remote hosts or cloud providers using volume drivers
  - Created through command or Docker can create a volume during container or service creation.
- Volume management commands:

  docker volume create

  docker volume prune

Option2: Bind Mounts (1/2)

- May be stored anywhere on the host system
  - may even be important system files or directories
  - Non-Docker processes on the Docker host or a Docker container can modify them at any time
- A file or directory on the host machine is mounted into a container
  - does not need to exist on the Docker host already
  - created on demand if it does not yet exist
- Can't use Docker CLI commands to directly manage bind mounts

Option2: Bind Mounts (2/2)

- Bind mounts may be stored anywhere on the host system
- Limited functionality compared to volumes
  - When you use a bind mount, a file or directory on the host machine is mounted into a container
  - The file or directory is referenced by its full or relative path on the host machine
- Bind mounts allow access to sensitive files
  - You can change the host filesystem via processes running in a container

Option 3: tmpfs (for Docker on Linux)

- Stored in the host system's memory only
  - never written to the host system's filesystem
- When the container stops, the `tmpfs` mount is removed, and files written there won't be persisted
- Unlike volumes and bind mounts
  - a `tmpfs` mount is temporary, and only persisted in the host memory
  - you can't share `tmpfs` mounts between containers.

Option 4: named pipe (for Docker on Win)

- `npipe` mount can be used for communication between the Docker host and a container
- Common use case is to run a third-party tool inside of a container and connect to the Docker Engine API using a named pipe

HOMEWORK

Step 1: Create volume

- docker volume --help
- docker volume create myvol-1
- docker volume ls

```
다운로드 — root@2a7dffdf9209: / — -zsh — 80×24
drsungwon@/Users/drsungwon/Downloads/:
$ docker volume --help
Usage:  docker volume COMMAND

Manage volumes

Commands:
  create      Create a volume
  inspect     Display detailed information on one or more volumes
  ls          List volumes
  prune       Remove unused local volumes
  rm          Remove one or more volumes

Run 'docker volume COMMAND --help' for more information on a command.
drsungwon@/Users/drsungwon/Downloads/:
$ docker volume create myvol-1

myvol-1
drsungwon@/Users/drsungwon/Downloads/:
$ docker volume ls
DRIVER    VOLUME NAME
local     myvol-1
drsungwon@/Users/drsungwon/Downloads/:
$
```

HOMEWORK

Step 2: Inspect created volume

- docker volume inspect myvol-1

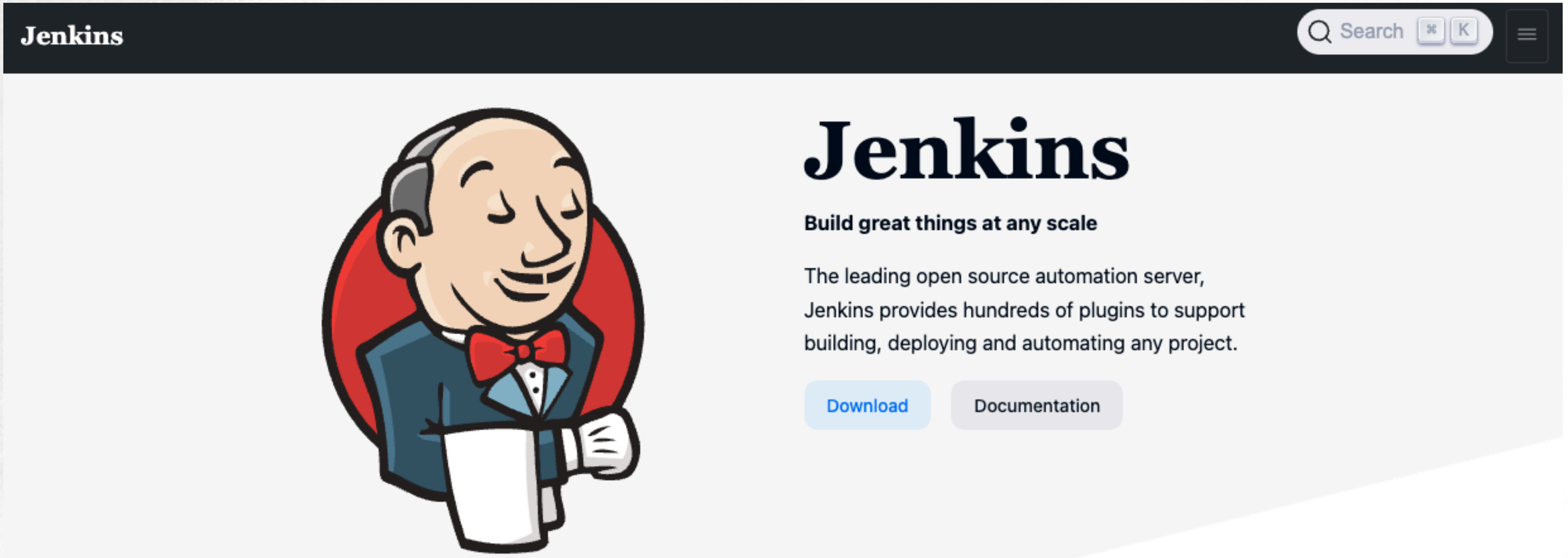
다운로드 — root@2a7dffdf9209: / — -zsh — 80×15

```
drsungwon@/Users/drsungwon/Downloads/:
$ docker volume inspect myvol-1
[
    {
        "CreatedAt": "2026-07-18T14:49:13Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/myvol-1/_data",
        "Name": "myvol-1",
        "Options": null,
        "Scope": "local"
    }
]
drsungwon@/Users/drsungwon/Downloads/:
$
```

🌟HOMEWORK🌟

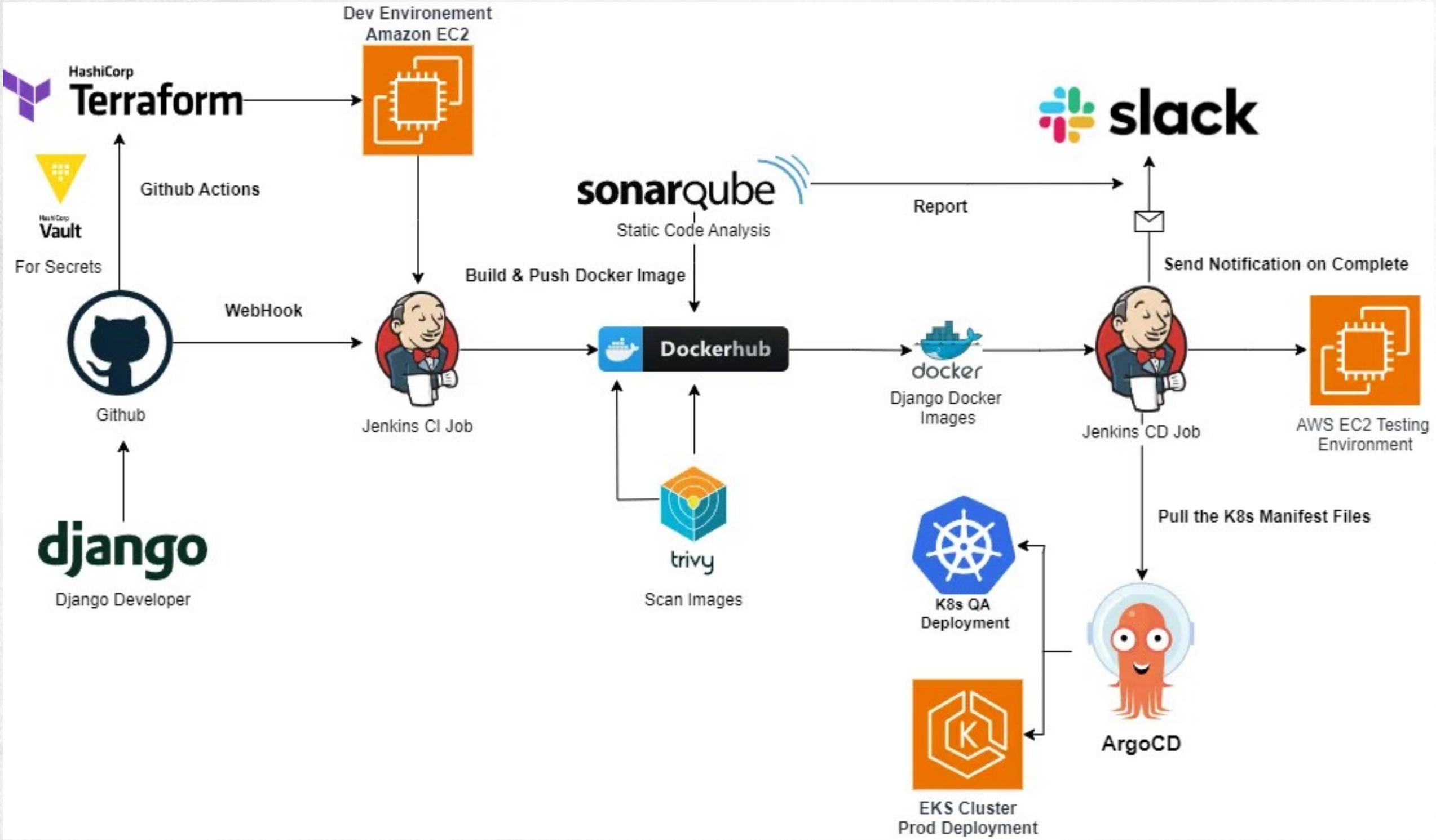
Step 3: Install Jenkins (1/2)

- Jenkins (https://jenkins.io/)
  - helps to automate the non-human part of the software development process
  - supports continuous integration and facilitating technical aspects of continuous delivery



FYI

Big Picture

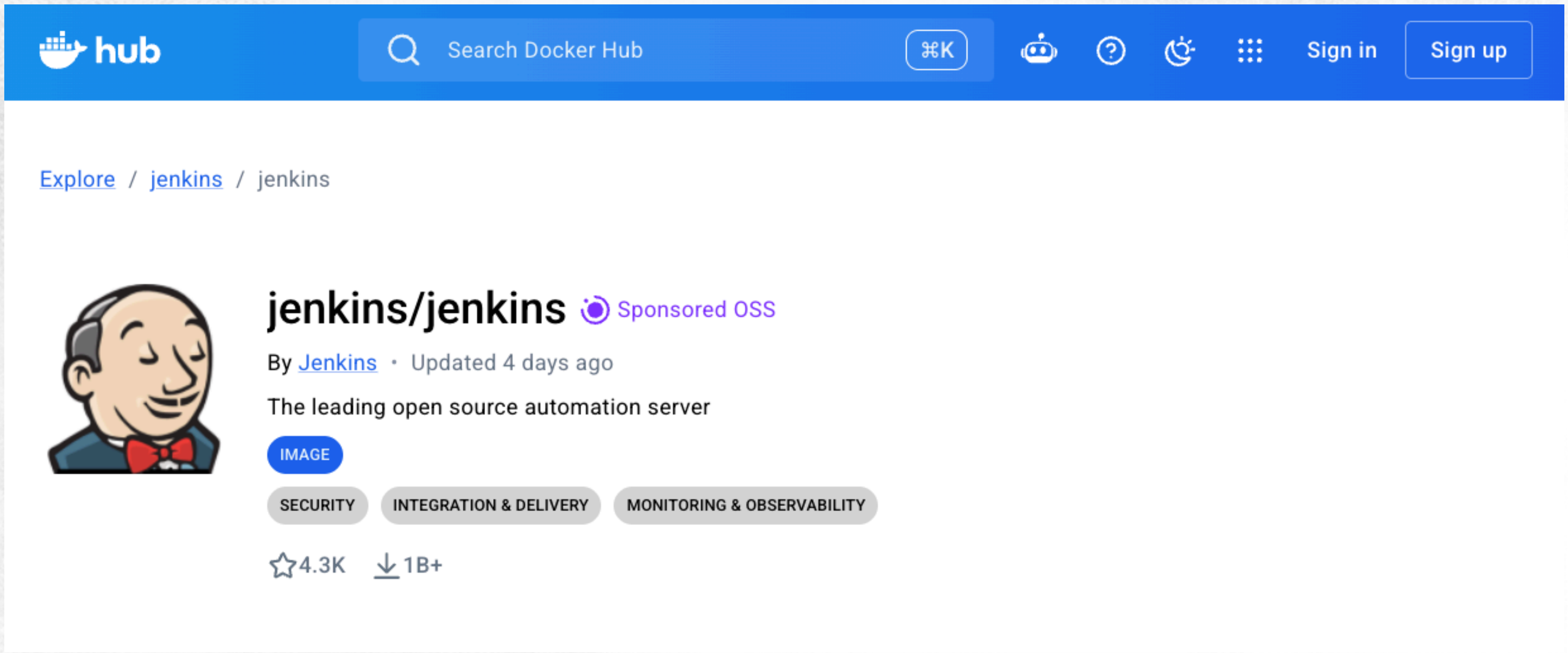


Practice

HOMEWORK

Step 3: Install Jenkins (2/2)

- docker pull jenkins/jenkins:lts



HOMEWORK

Step 4: Execute Jenkins with Volume

docker run -p 8080:8080 -p 50000:50000 -v myvol-1:/var/jenkins_home jenkins/jenkins:lts

How to use this image

```
docker run -p 8080:8080 -p 50000:50000 jenkins
```

This will store the workspace in /var/jenkins_home. All Jenkins data lives in there - including plugins and configuration. You will probably want to make that a persistent volume (recommended):

```
docker run -p 8080:8080 -p 50000:50000 -v /your/home:/var/jenkins_home jenkins
```

This will store the jenkins data in `/your/home` on the host. Ensure that `/your/home` is accessible by the jenkins user in container (jenkins user - uid 1000) or use `-u some_other_user` parameter with `docker run`.

HOMEWORK

Step 5: Confirm Installation (1/2)

```
[LF]>
[LF]> *************************************************************
[LF]> *************************************************************
[LF]> *************************************************************
[LF]>
[LF]> Jenkins initial setup is required. An admin user has been created and a password generated.
[LF]> Please use the following password to proceed to installation:
[LF]>
[LF]> 731560ac93ba4cbab4d3e05f8cd98474
[LF]>
[LF]> This may also be found at: /var/jenkins_home/secrets/initialAdminPassword
[LF]>
[LF]> *************************************************************
[LF]> *************************************************************
[LF]> *************************************************************
```

HOMEWORK

Step 5: Confirm Installation (2/2)

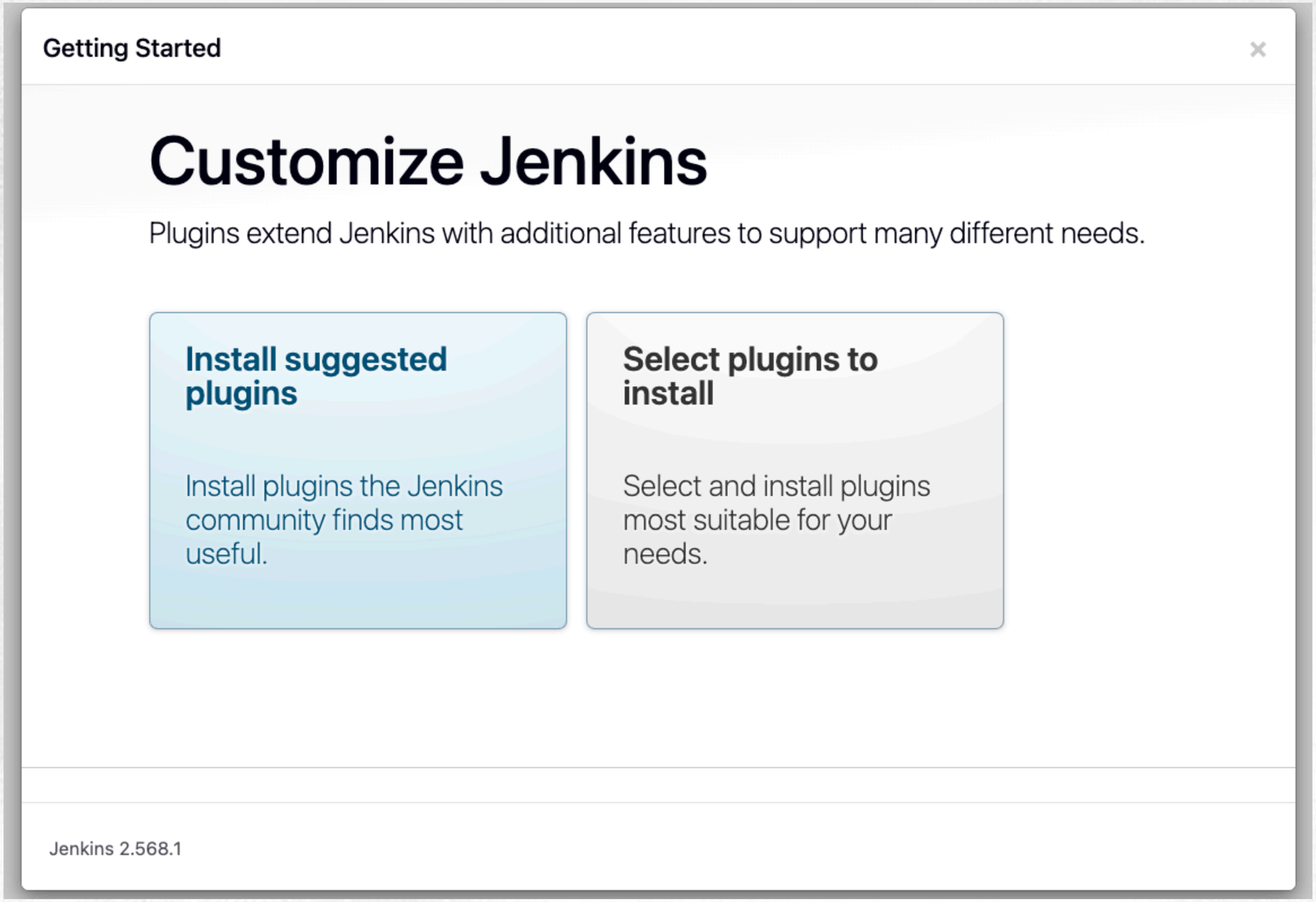
- Connect to http://localhost:8080
- Input installation password & press “Continue”



HOMEWORK

Step 6: Customize Jenkins (1/2)

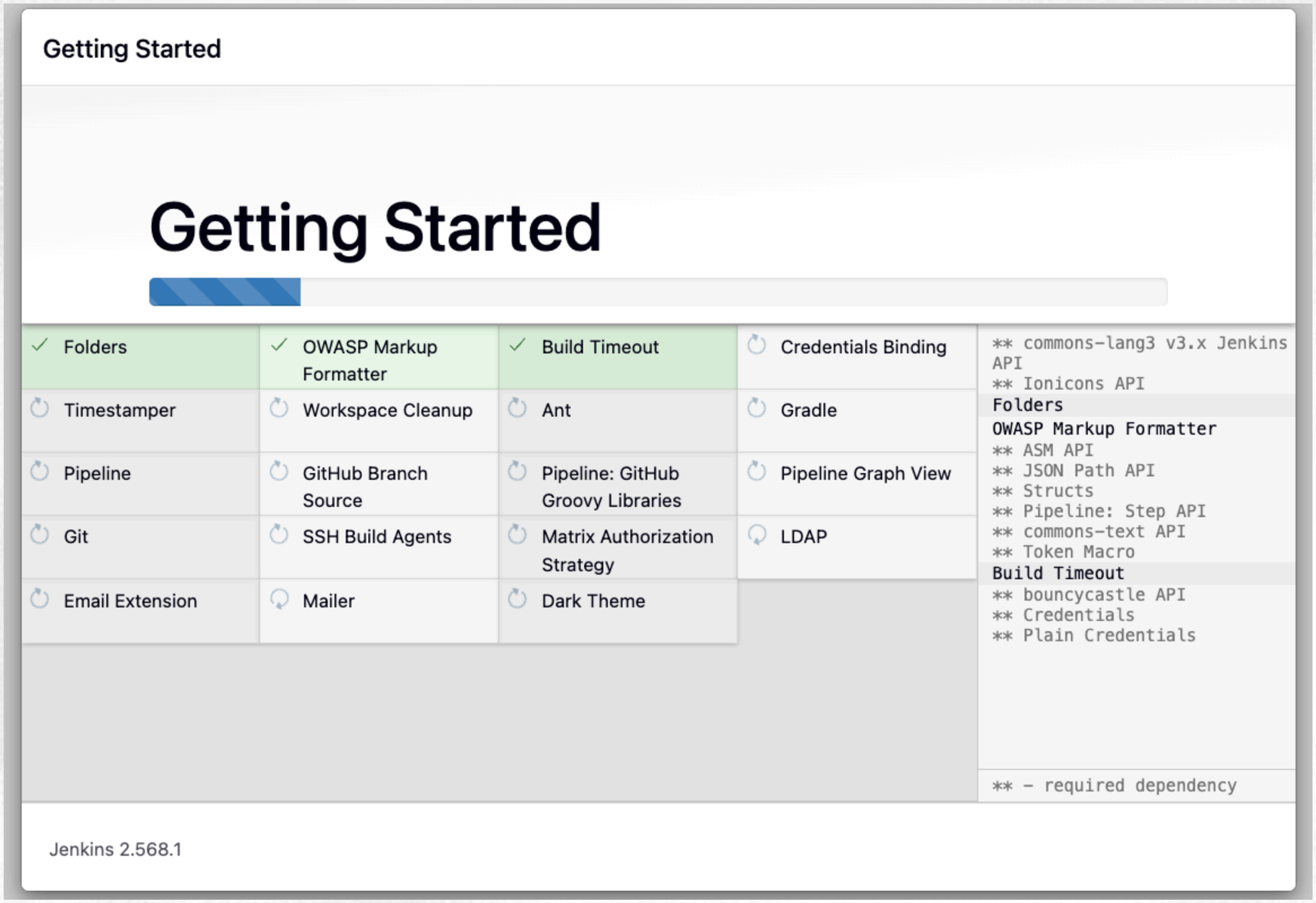
- Select “Install suggested plugins” & wait …



HOMEWORK

Step 6: Customize Jenkins (2/2)

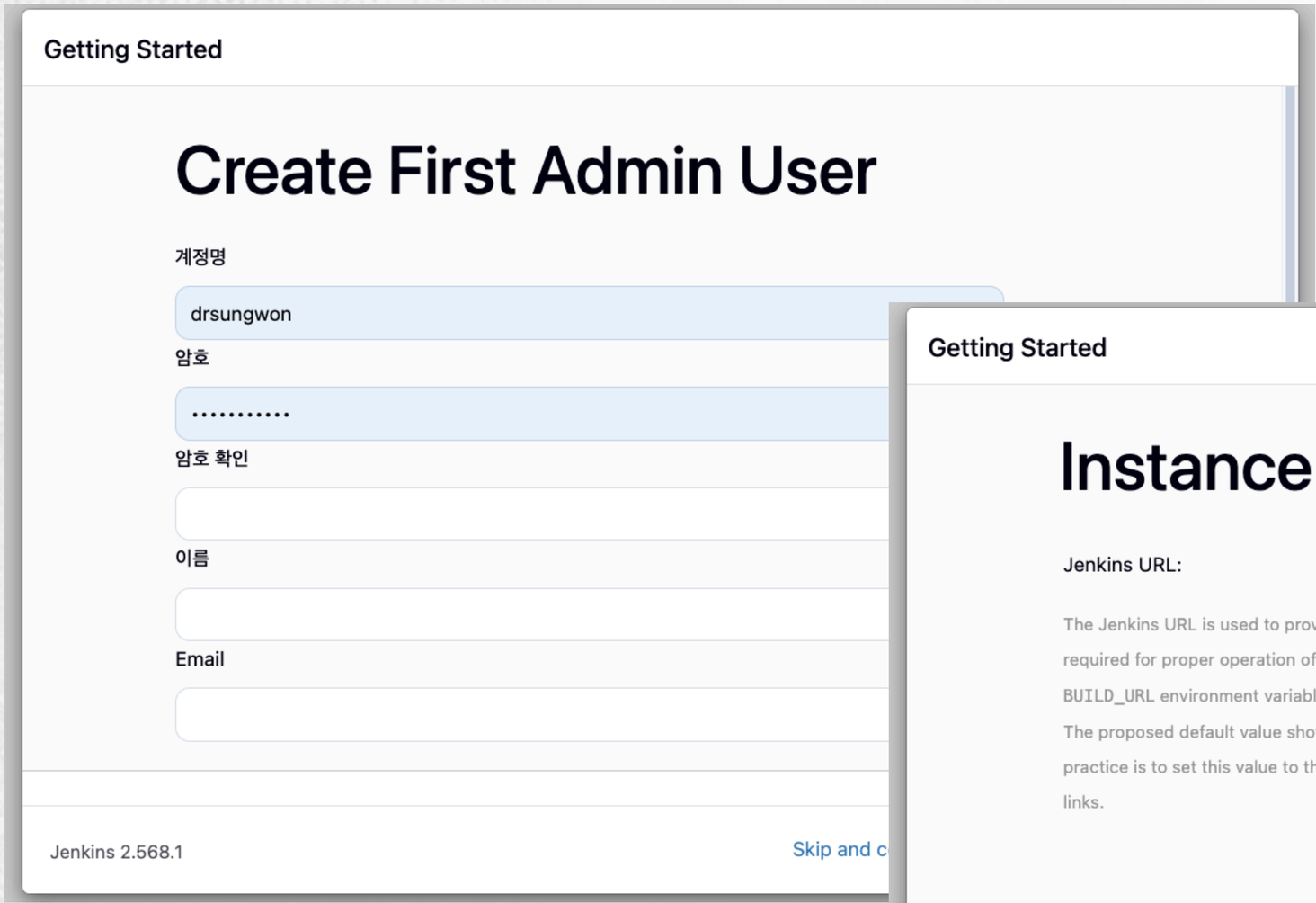
- Select “Install suggested plugins” & wait …

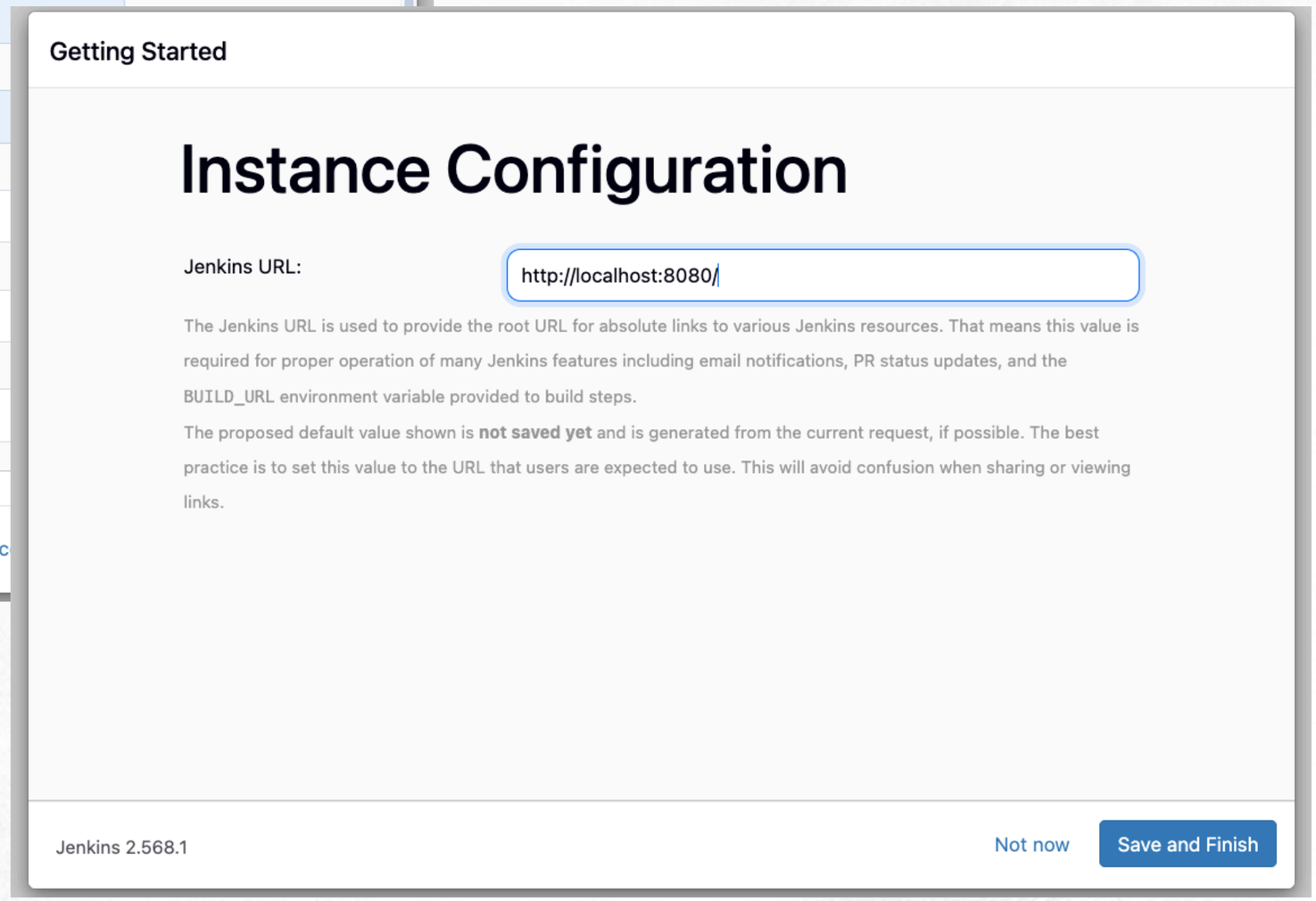


HOMEWORK

Step 7: Create Admin User

- Input information & select “Save and Finish”

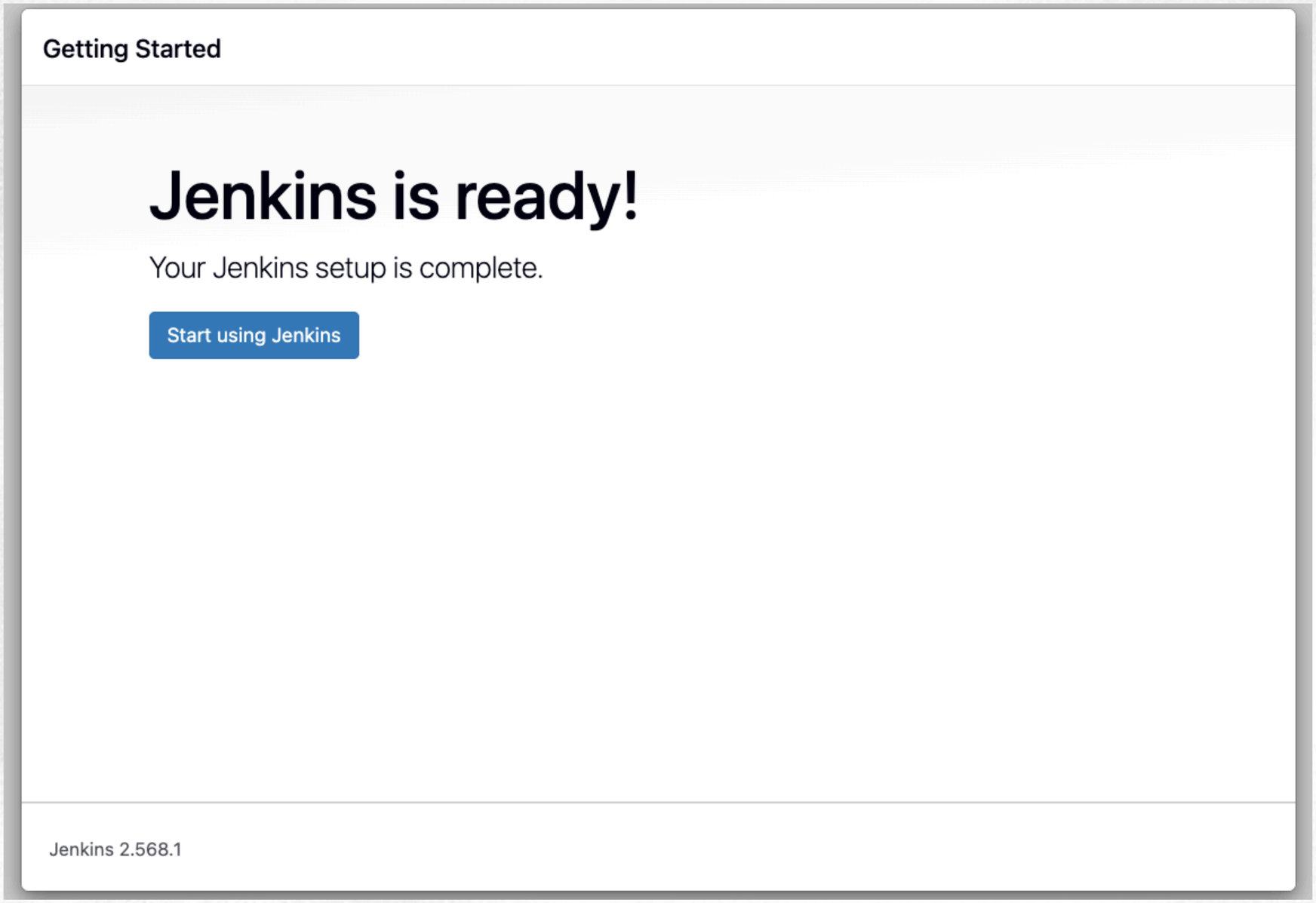




HOMEWORK

Step 8: Confirm Installation

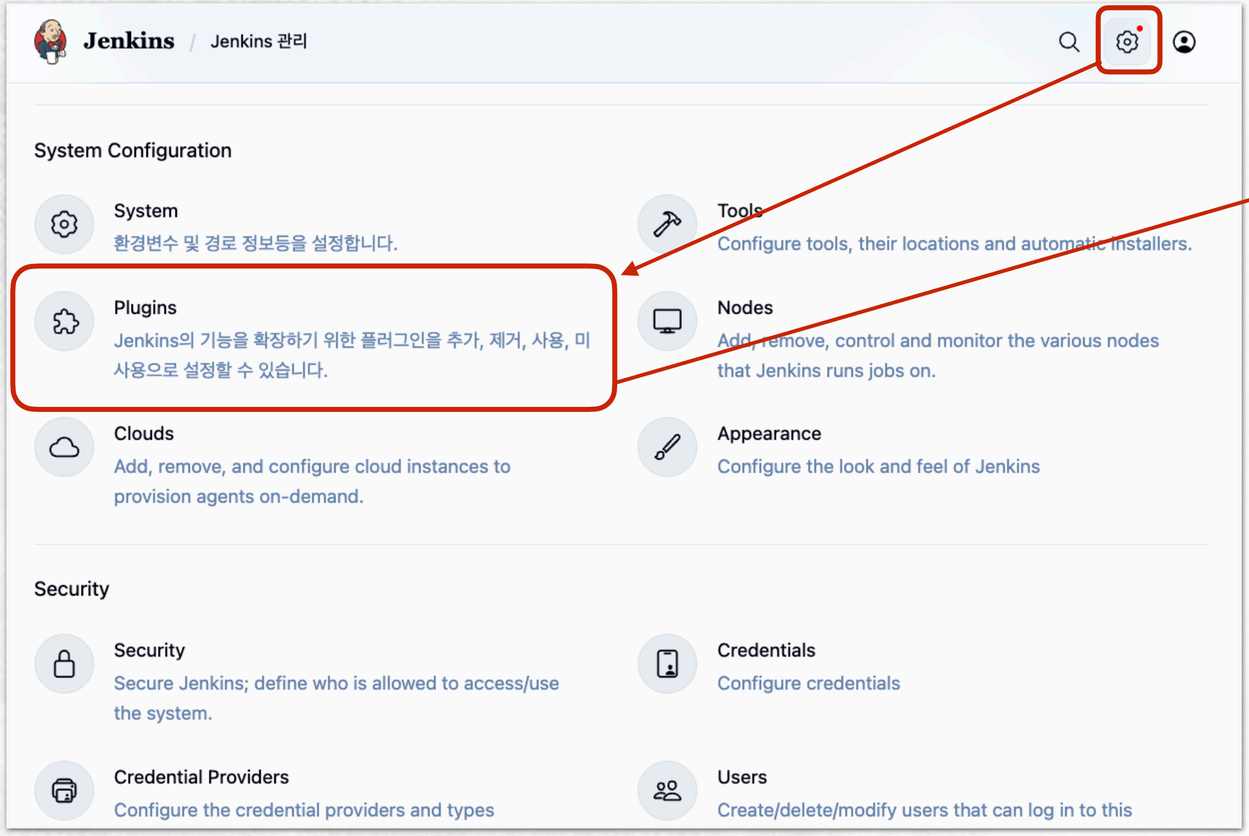
- Input information & select “Save and Finish”

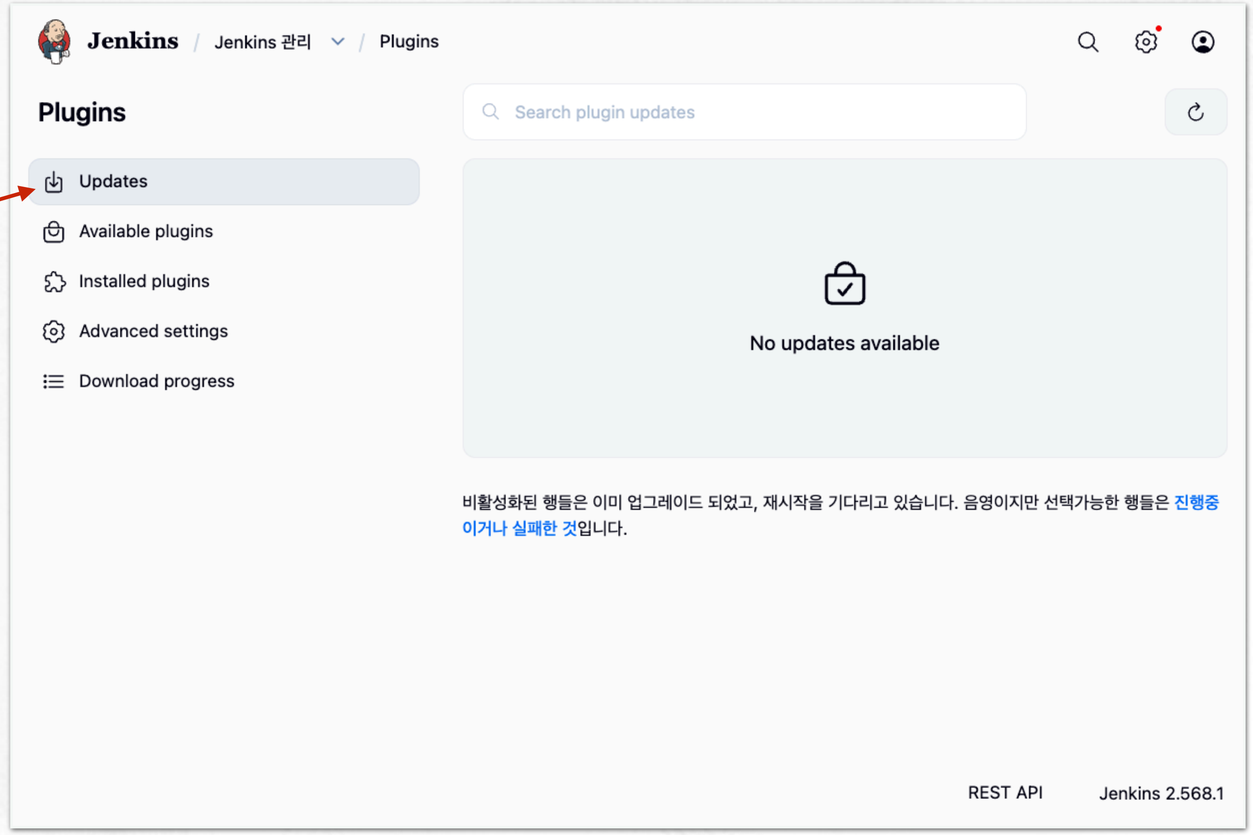


HOMEWORK

Step 9: Update Jenkins

- Through “Jenkins 관리"





HOMEWORK

Step 10: Jenkins Upgrade (1/2)

- Signout jenkins
- docker container stop {jenkins container ID}
- docker volume ls
- docker volume inspect myvol-1

```
다운로드 — root@2a7dffdf9209: / — -zsh — 80×19
drsungwon@/Users/drsungwon/Downloads/:
$ docker volume ls
DRIVER    VOLUME NAME
local     myvol-1
drsungwon@/Users/drsungwon/Downloads/:
$ docker volume inspect myvol-1
[
    {
        "CreatedAt": "2026-07-18T14:49:13Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/myvol-1/_data",
        "Name": "myvol-1",
        "Options": null,
        "Scope": "local"
    }
]
drsungwon@/Users/drsungwon/Downloads/:
$
```

HOMEWORK

Step 10: Jenkins Upgrade (2/2)

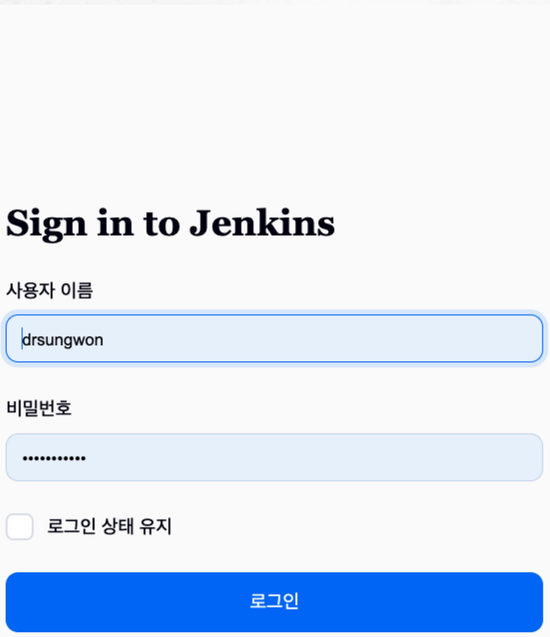
- docker run --name jenkins-production ₩
  --detach ₩
  -p 50000:50000 ₩
  -p 8080:8080 ₩
  -v myvol-1:/var/jenkins_home ₩
  jenkins/jenkins:lts

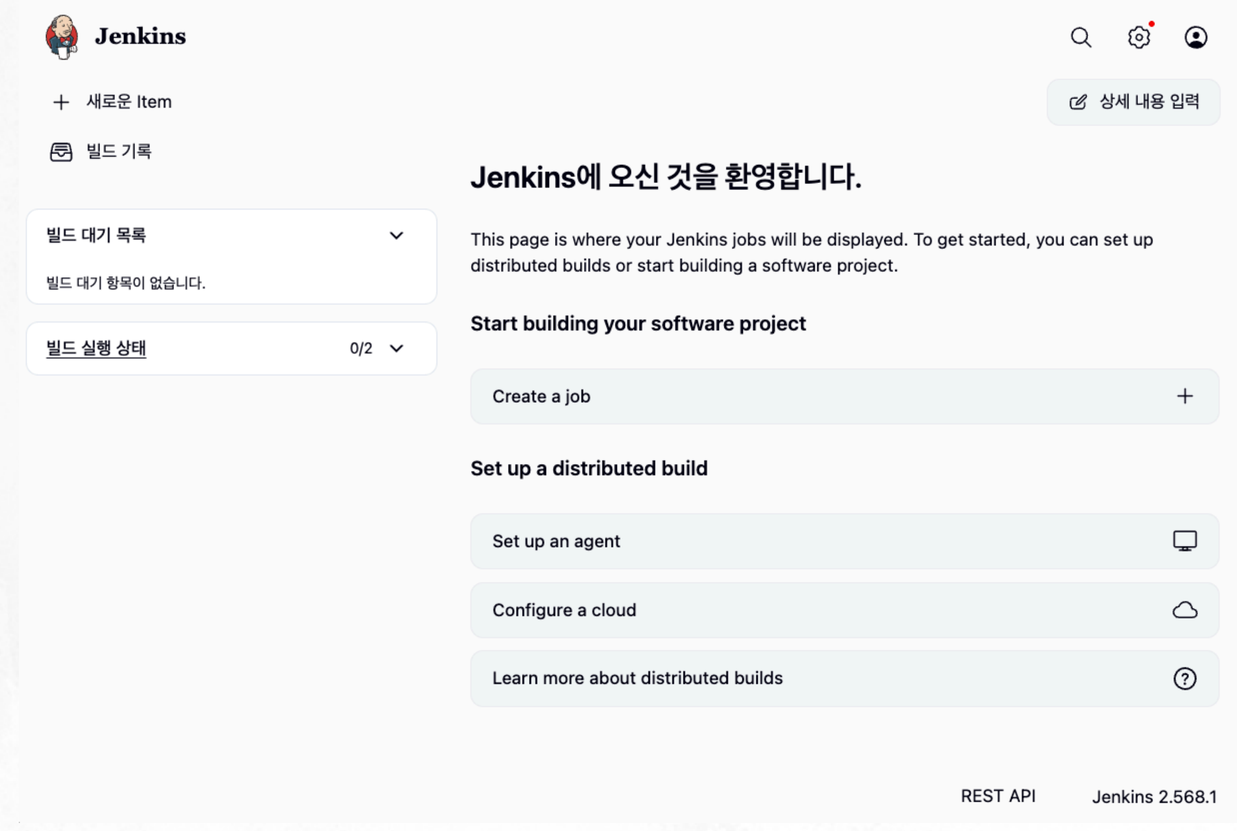
HOMEWORK

Step 11: Execute Jenkins Again

- Access to http://localhost:8080
- Check Jenkins Status



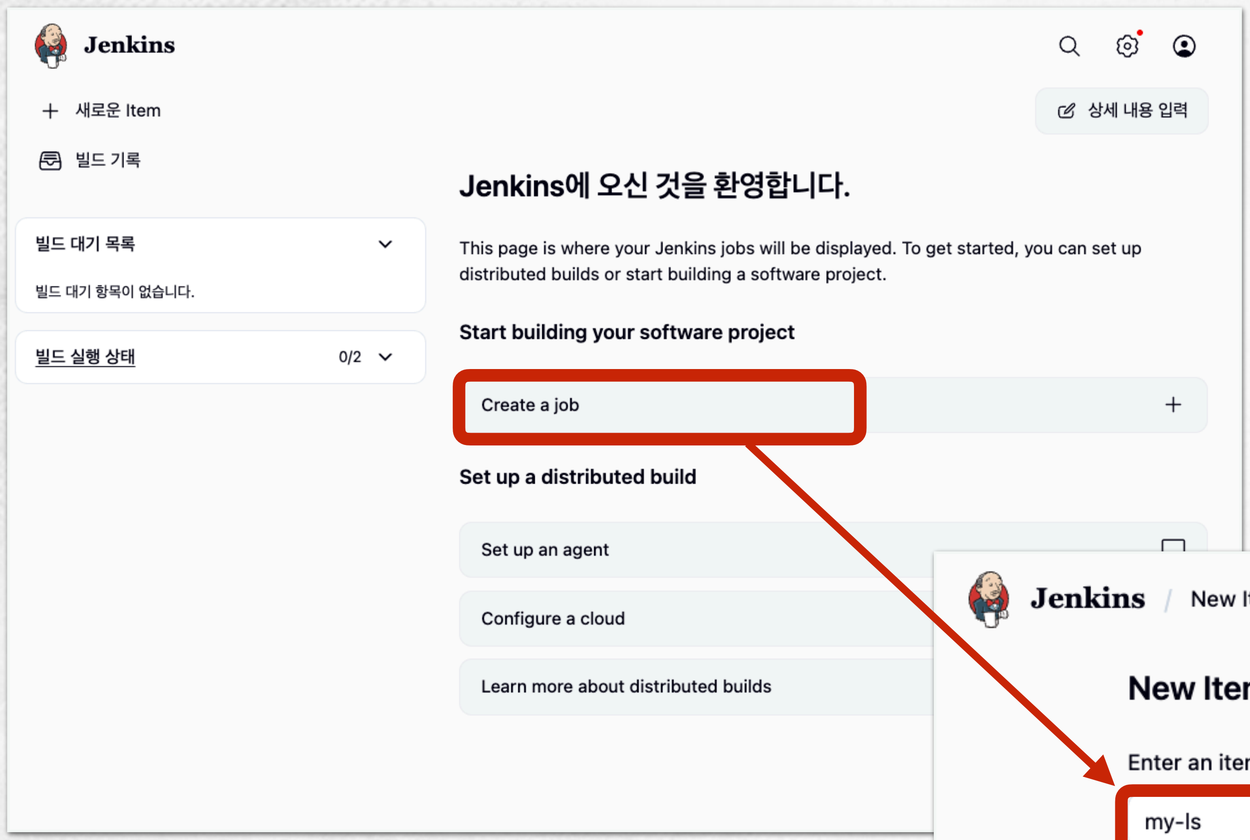


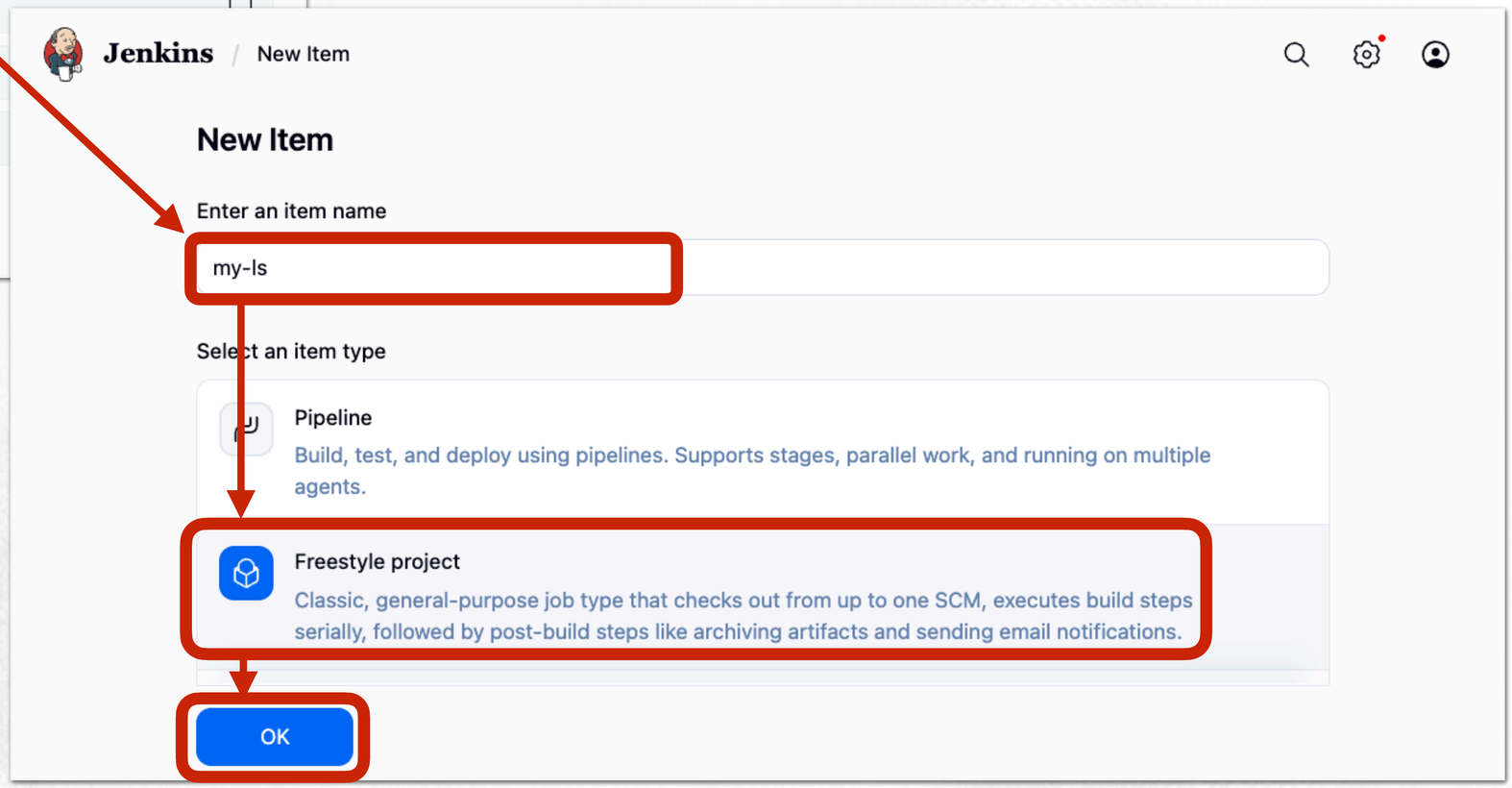


HOMEWORK

Step 12: Create & Execute Job (1/4)

- Select “새 작업” and input 1st job name (ex: my-ls)

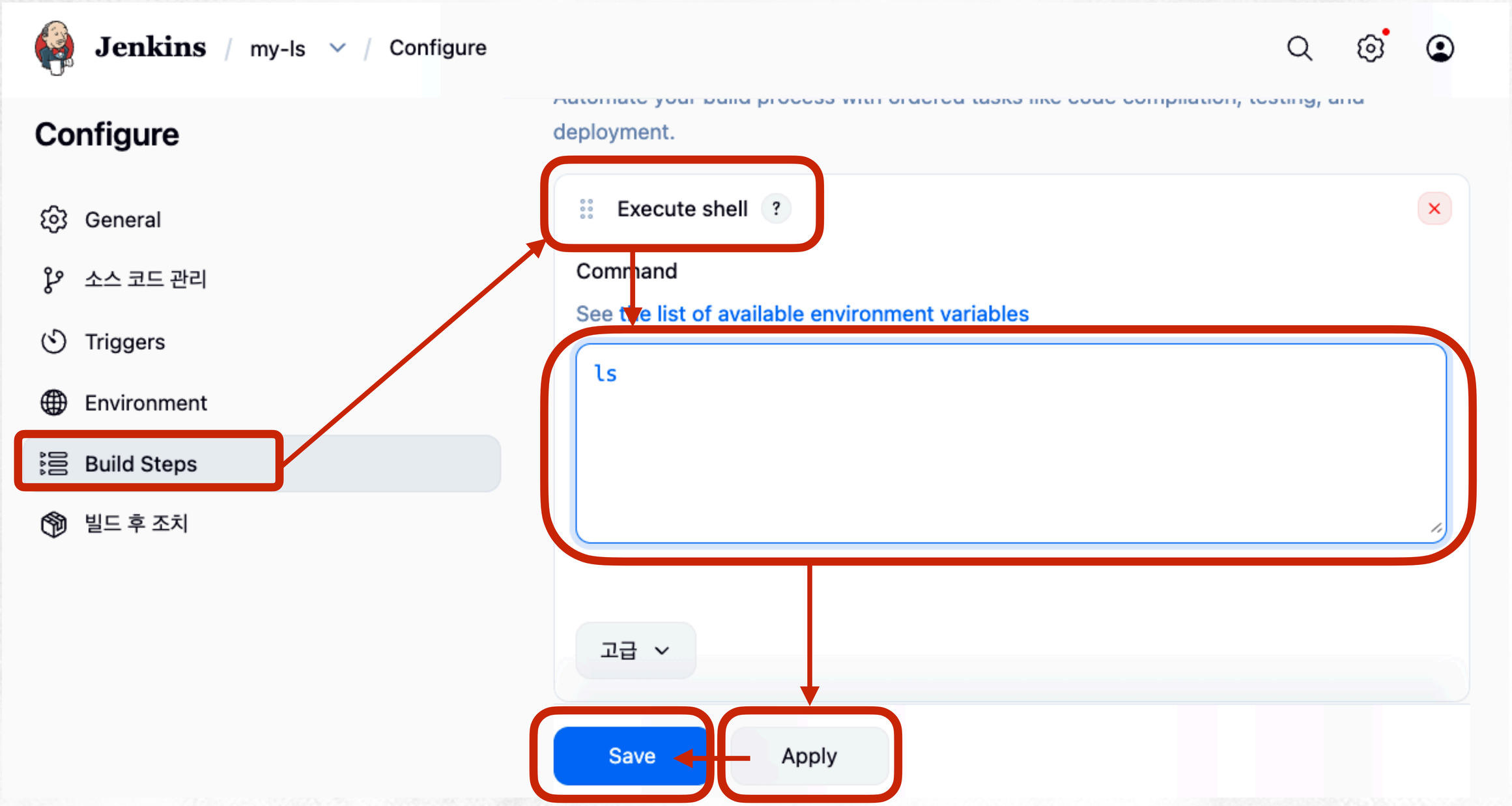




HOMEWORK

Step 12: Create & Execute Job (2/4)

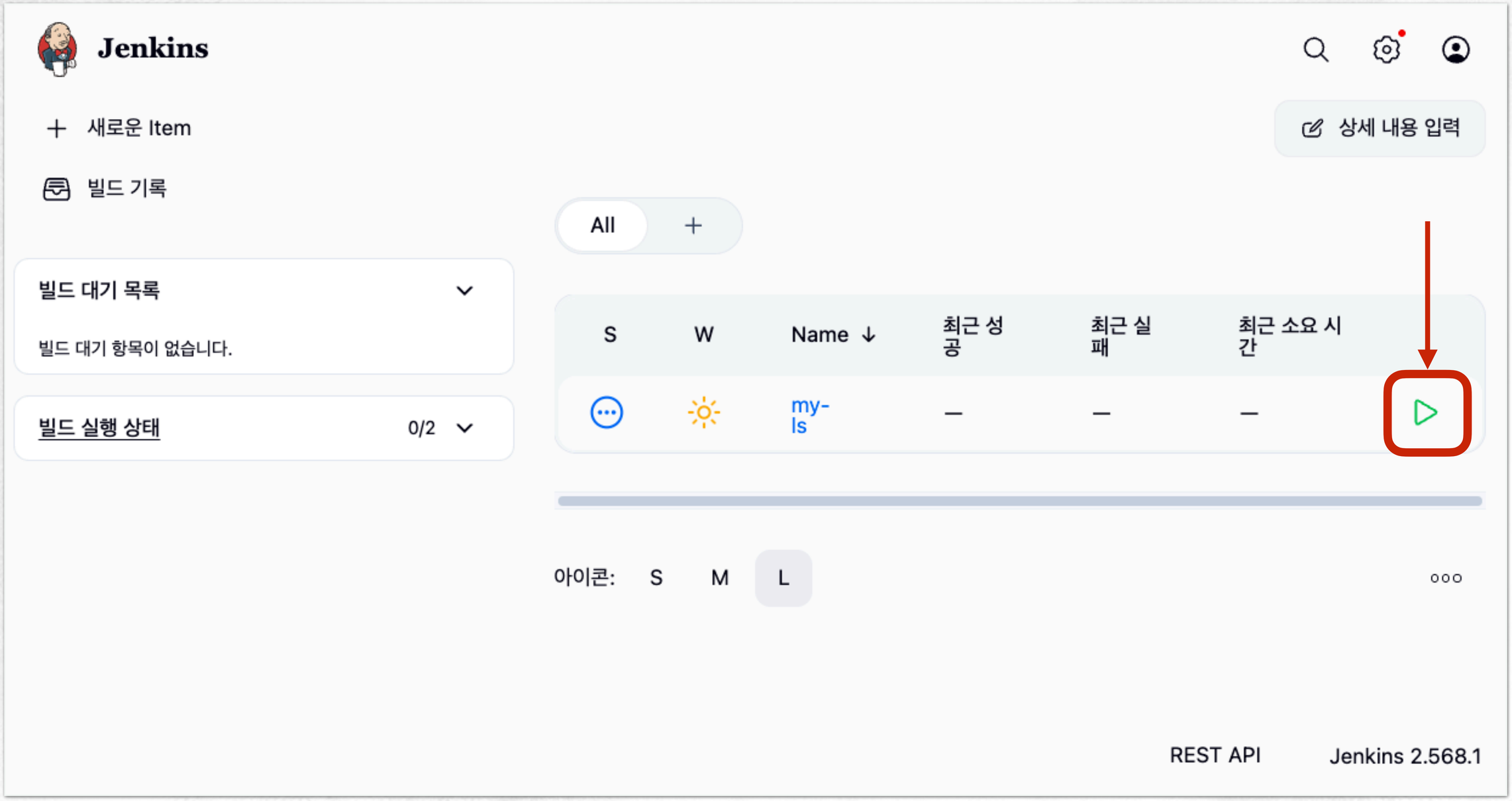
- Select “Build”, and select “Execute shell”
- Type “ls” command, then select “Apply” & “Save”



HOMEWORK

Step 12: Create & Execute Job (3/4)

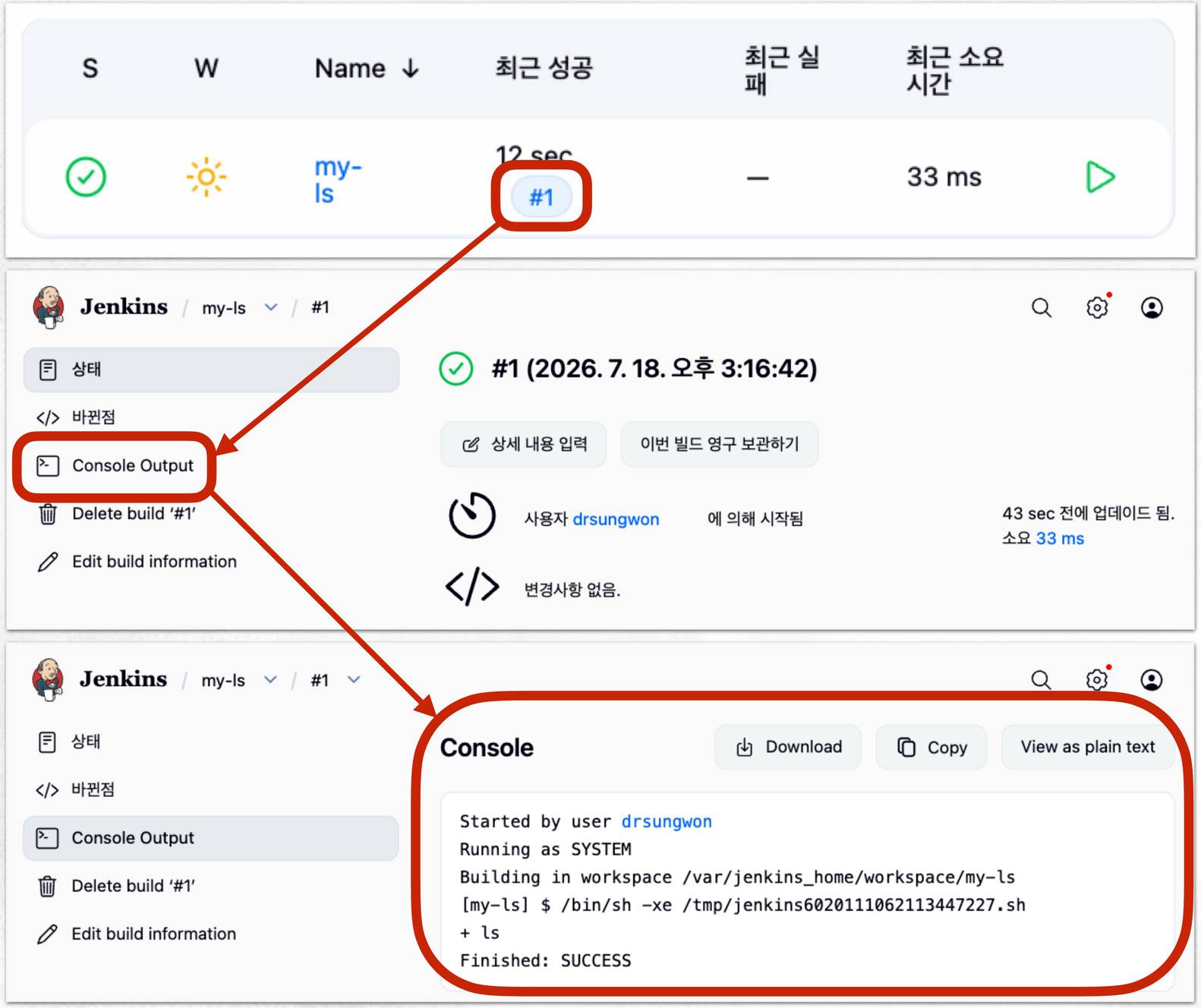
- Go to top menu, and check “my-ls” job
- No recent success job? Then execute 1st job now!



HOMEWORK

Step 12: Create & Execute Job (4/4)

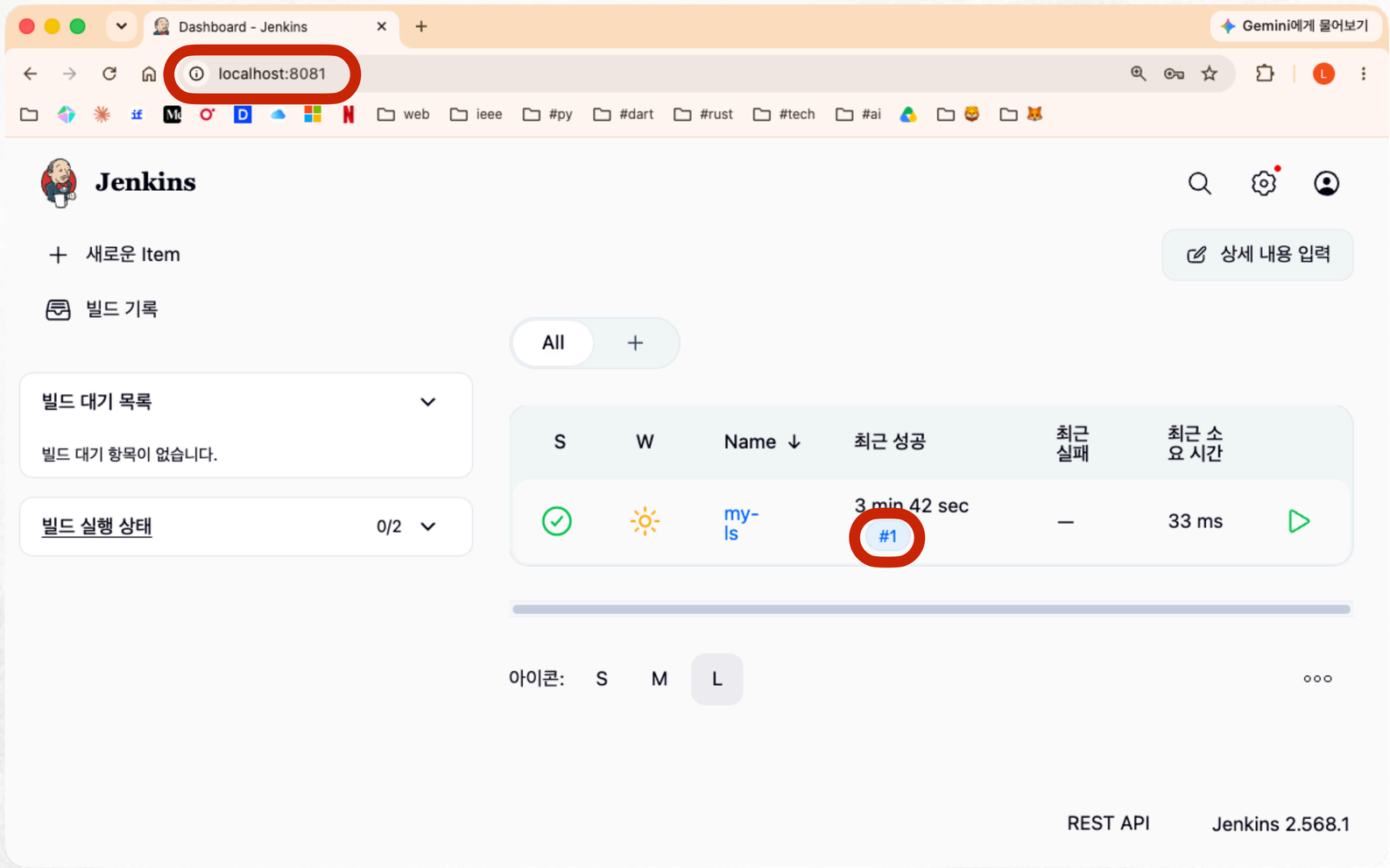
- Check recent job result



HOMEWORK

Step 13: Execute another Jenkins

- With the same volume
- docker run --name jenkins-production-copy ₩
  --detach ₩
  -p 50001:50000 ₩
  -p 8081:8080 ₩
  -v myvol-1:/var/jenkins_home ₩
  jenkins/jenkins:lts



HOMEWORK

Step 14: Remove Volume

- docker volume ls
- docker volume rm {volume name}
- docker volume prune

Another Approach?

- docker compose up
- Check your “./jenkins_home” for Container files

docker-compose.yml

```yaml
version: '3.7'
services:
  manager:
    image: jenkins/jenkins:lts
    ports:
      - 8080:8080
    volumes:
      - ./jenkins_home:/var/jenkins_home
```

Thank you

경희대학교
KYUNG HEE UNIVERSITY